

#作业四

##1. 请阐述 C++ 中动态绑定和静态绑定的概念，并说明在什么情况下会发生动态绑定。

- 静态绑定
 - 静态绑定是在编译时刻根据对象的类型来决定采用哪一个消息处理函数。在非虚函数、普通函数调用、对具体类类型的对象的方法调用的情况下使用。
- 动态绑定
 - 动态绑定是在运行时进行的绑定，主要发生在通过基类的指针或引用调用虚函数时；在运行时根据对象的实际类型（而不是指针或引用的类型）来确定调用哪个函数或方法。当调用通过指针或引用调用的虚函数时、实现多态、继承和覆盖时的情况使用。

##2. 为什么析构函数一般声明为虚函数

- 便于在派生类与基类的变量有差异时，能够通过不同的析构函数释放内存;否则的话，只有基类的析构函数会被调用，可能导致内存泄漏

##3.为什么基类的构造函数和析构函数中对虚函数的调用不发生动态绑定

- 确保当造一个派生类对象时，构造过程从基类开始，然后逐步向下到派生类；这样的话，对象的类型会被视为基类类型，而不是派生类类型，避免在基类的构造函数中调用一个虚函数时会派发到派生类中重写的版本，产生派生类中元素错误创建和访问的问题。

##4. 什么是抽象类，抽象类的作用是什么，抽象类的派生类是否一定要给出纯虚函数的实现？

- - 纯虚函数是没给出实现的虚函数，函数体用“=0”表示；包含纯虚函数的类称为抽象类。
 - 抽象类的作用是为派生类提供一个基本框架和一个公共的对外接口
 - 不一定，派生类可以有选择的实现有需要的抽象类中的纯虚函数，不是每个纯虚函数都会被每一个派生类实现

##5. 使用类聚合的方式和类组合的方式复用代码有什么不同?什么情况下适合使用聚合?什么情况下适合使用组合?在编程时需要注意什么？

- - 二者都是类之间的关系，区别在于聚合关系中，被包含的对象与包含它的对象独立创建和消亡，被包含的对象可以脱离包含它的对象独立存在；组合关系中，被包含的对象随包含它的对象创建和消亡，被包含的对象不能脱离包含它的对象独立存在。
 - 聚合适用于那些需要表示整体与部分关系，但部分可以独立于整体存在的情况。
 - 组合适用于那些部分与整体生命周期相同的情况。

##6. 聚合/组合相比继承的代码复用有哪些优点?能否仅仅通过前两者 实现代码复用?为什么？

- - 继承与封装存在矛盾，聚合/组合与封装则不存在这个矛盾；聚合/组合仅需要public一种接口，而继承需要private、public等多种接口
 - 具有聚合/组合关系的两个类不具有子类型关系，因此不便于代码互用

##7. 多继承解决了什么问题?额外引入了哪些问题?这些问题如何解决？

- - 多继承允许一个类从多个基类继承属性和方法，允许一个类实现多个接口，便于实现同属于两个基类的派生类的实现

- 多继承引入的问题：两个基类继承自同一个祖先类，然后另一个类又从这两个基类继承时，就会产生菱形继承结构，引发歧义；同时多继承使类的设计和维持变得更加复杂
- 解决这些问题的方法：使用虚基类可以解决菱形继承问题，同时注意设计时的合理性，仅在必要的时候使用多继承

##8. 动物园管理系统

```
class Animal {
    protected: //private
        std::string name;
        int age;
        std::string color;

    public:
        Animal(std::string n, int a, std::string c) : name(n), age(a),
color(c) {}

        virtual void makeSound() = 0;
        virtual void animal_information() = 0;
};

class Cat : public Animal {
    public:
        Cat(std::string n, int a, std::string c) : Animal(n, a, c) {}

        void makeSound() override {
            std::cout << name << " says: miaomiaomiao!" << std::endl;
        }
        void animal_information() override {
            std::cout << " The name of the little" << color << "cat is:
"<<name <<std::endl;
            std::cout <<"And it is "<<age <<"years old!"<<std::endl;
        }
};

class Dog : public Animal {
    public:
        Dog(std::string n, int a, std::string c) : Animal(n, a, c) {}

        void makeSound() override {
            std::cout << name << " says: wangwangwang!" << std::endl;
        }
        void animal_information() override {
            std::cout << " The name of the little" << color << "dog is:
"<<name <<std::endl;
            std::cout <<"And it is "<<age <<"years old!"<<std::endl;
        }
};
```